

Relatório do Projeto Final

Luiz Gustavo Turatti

Fase 4 - Qualidade e Testes para o EventMaster

Esta quarta etapa concentra-se em converter o protótipo inicial em uma solução robusta de nível de produção, com foco em sustentabilidade, testabilidade e extensibilidade.

Objetivos Principais

- Aplicar de forma rigorosa os princípios SOLID.
- Implementar um Padrão de Projeto GoF (*Gang of Four*) adequado.
- Demonstrar a adoção de boas práticas de *Clean Code*.
- Construir testes unitários utilizando a metodologia TDD.
- Evidenciar a cobertura de testes de comportamento via BDD.
- Elevar os índices de manutenibilidade e escalabilidade do sistema.

1. Serviço Selecionado para Refatoração

O microsserviço escolhido para passar pelo processo de refatoração foi o **Serviço de Pagamento** (*Payment Service*). Este componente foi considerado ideal devido às seguintes características:

- Concentra regras de negócio críticas e complexas.
- Demanda suporte a múltiplos métodos de pagamento.
- Possui alta necessidade de extensibilidade.
- Trata-se de uma camada onde a segurança é um fator crítico.
- Depende frequentemente de integrações com parceiros externos.

2. Objetivos Arquiteturais

O Serviço de Pagamento foi remodelado para suportar nativamente:

- Cartão de Crédito
- PIX
- PayPal
- Inclusão modular de futuros provedores de pagamento

Premissa: Toda essa flexibilidade é alcançada **sem alterar a lógica de negócio central** do microsserviço. Esse desacoplamento valida e demonstra diretamente a aplicação de dois conceitos:

- O Princípio Aberto/Fechado (*Open/Closed Principle* - OCP).
- O Padrão de Projeto *Strategy*.

3. Aplicação Prática dos Princípios SOLID

S - Princípio da Responsabilidade Única (SRP)

Cada classe do ecossistema assume estritamente uma única responsabilidade funcional.

Classe	Responsabilidade Principal
PaymentService	Coordena o fluxo de execução do pagamento.
CreditCardPayment	Processa pagamentos específicos de cartões de crédito.
PixPayment	Processa pagamentos específicos via PIX.
PaymentValidator	Centraliza as regras de validação cadastral e financeira.
PaymentRepository	Abstrai as operações de persistência de dados.

O - Princípio Aberto/Fechado (OCP)

Novas modalidades de pagamento podem ser acopladas à arquitetura sem que haja necessidade de modificar o código-fonte existente. Como exemplo prático, é possível incluir o método `ApplePayPayment` sem realizar qualquer modificação na classe `PaymentService`.

L - Princípio da Substituição de Liskov (LSP)

Todas as implementações concretas de estratégias de pagamento podem substituir sua interface base sem quebrar o comportamento esperado da aplicação. Exemplo: `PaymentStrategy strategy = new PixPayment();`.

I - Princípio da Segregação de Interfaces (ISP)

O design do sistema prioriza a criação de interfaces enxutas, modulares e altamente focadas em seus escopos.

D - Princípio da Inversão de Dependência (DIP)

Os módulos de alto nível passam a depender exclusivamente de abstrações, mitigando o acoplamento com classes concretas. Nesse modelo, o `PaymentService` depende unicamente da **interface `PaymentStrategy`**.

4. Padrão de Projeto GoF Adotado

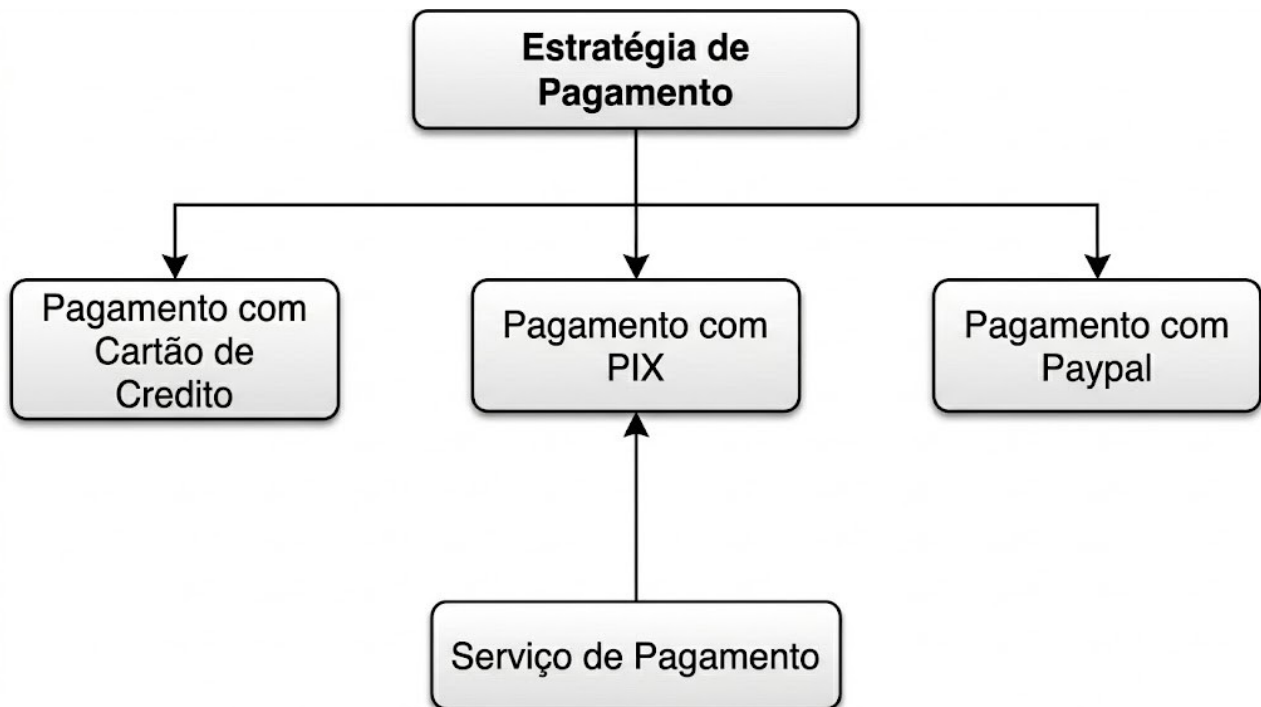
A escolha pelo Padrão Strategy mostrou-se ideal pelas seguintes razões do domínio:

- As regras de cobrança variam de acordo com o método.
- Os provedores de adquirência possuem fluxos distintos.
- As rotinas de análise de fraude mudam conforme o canal de entrada.
- As APIs externas possuem contratos de integração heterogêneos.

Dessa forma, o padrão encapsula cada algoritmo de pagamento de maneira isolada e independente.

5. Arquitetura do Padrão Strategy

O diagrama abaixo ilustra o fluxo de dependências e a hierarquia estabelecida na implementação:



6. Princípios de Clean Code Aplicados

Princípio	Aplicação Prática na Solução
Métodos Pequenos	Funções concisas e focadas em uma única ação específica.
Nomes Significativos	Classes, variáveis e métodos com nomenclaturas autoexplicativas.
Baixo Acoplamento	Uso sistemático de interfaces para reduzir a dependência direta entre componentes.
Alta Coesão	Componentes com responsabilidades lógicas estreitamente delimitadas. Módulos focados exclusivamente em seu domínio de negócio.
Imutabilidade	Proteção e integridade de dados garantidas através de DTOs (<i>Data Transfer Objects</i>) protegidos contra efeitos colaterais.
Código Não Duplicado	Centralização de lógicas comuns em abstrações reutilizáveis.
Validação Defensiva	Proteção da integridade do sistema por meio de validações rigorosas na entrada de dados. Tratamento antecipado de estados inválidos com falhas rápidas (<i>Fail-Fast</i>).

7. Protótipo em Java — SOLID + Padrão Strategy

- Requisições de pagamento
- Estratégias de pagamento
- Pagamento com cartão de crédito
- Pagamento com PIX
- Pagamento com Paypal
- Validador de pagamentos
- Serviços de pagamentos

8. TDD — Desenvolvimento Orientado por Testes

O design da solução foi concebido em conformidade com as práticas do TDD, seguindo o fluxo iterativo tradicional:

1. **RED:** Escrita estruturada de um teste automatizado que falha devido à ausência da implementação.
2. **GREEN:** Codificação do trecho mínimo necessário para que o teste passe com sucesso.
3. **REFACTOR:** Refatoração e aprimoramento do design do código sob a proteção da suíte de testes.

9. Diretrizes de Cobertura Unitária

A lógica comportamental cobre com rigidez os seguintes cenários de negócio:

- Execução de pagamentos em condições ideais (fluxo principal)
- Rejeição preemptiva de transações com valores negativos ou zerados
- Bloqueio de requisições com e-mail de cliente ausente ou em branco
- Validação do acionamento correto e isolado das estratégias escolhidas

10. Suíte de Testes Unitários com Junit

Código em Java

11. Pirâmide de Testes

A governança e a estratégia de validação do EventMaster apoiam-se rigorosamente no modelo da Pirâmide de Testes:



12. Características dos Testes Unitários

- Tempo de resposta extremamente ágil e baixo consumo de recursos
- Totalmente isolados de dependências externas
- Comportamento estritamente determinístico
- Amplo emprego de dublês de testes (*mocks* e *stubs*)
- *Exemplos*: Regras de validação de pagamentos, rotinas de geração de JWT, motores de cálculo de descontos e lógicas de reserva de estoque.

13. Testes de Integração

- Objetivo central: Garantir o alinhamento de contratos e a integridade da comunicação entre componentes distribuídos.
- *Exemplos de escopo*: Integração entre Serviço de Vendas e Serviço de Pagamentos; fluxo entre API Gateway e Serviço de Autenticação; validação de acoplamento entre Produtores e Consumidores Apache Kafka.

14. Testes Ponta a Ponta (*End-to-End* - E2E)

- Objetivo central: Mapear e certificar o sucesso da experiência integral e da jornada do usuário na plataforma.
- *Exemplos de fluxos*: Autenticação no sistema $\rightarrow$ Adição de itens ao carrinho $\rightarrow$ Finalização e compra do ingresso $\rightarrow$ Recebimento de notificações de confirmação.

15. BDD — Testes Baseados em Comportamento

O BDD foi introduzido para unificar a linguagem entre o time técnico e as partes interessadas de negócio, descrevendo funcionalidades sob a perspectiva do usuário. A tecnologia homologada para esta camada é o **Cucumber**.

16. Cenário BDD Proposto

Funcionalidade: Pagamento de Ingressos

Cenário: Liquidação bem-sucedida via cartão de crédito

- Dado que o cliente realizou a seleção dos ingressos válidos
- E o valor consolidado da transação é de 200.00
- Quando o cliente optar por efetuar o pagamento via cartão de crédito
- Então o pagamento deve ser processado e aprovado com sucesso
- E o pedido correspondente deve ser confirmado no sistema

17. Implementação dos Passos (*Step Definitions*)

Código em Java

18. Métricas de Qualidade Homologadas

Métrica Alvo	Meta Estabelecida
Cobertura de Código Unitário	\$> 80\%\$
Cobertura em Serviços Críticos	\$> 90\%\$
Análise Estática de Código	Aprovado sem restrições no SonarQube
Complexidade Ciclomática	Mantida em níveis estritamente baixos
Débito Técnico	Monitoramento e mitigação contínuos
Varredura de Segurança (<i>Security Scan</i>)	Verificação obrigatória por execução (Gate)

19. Ferramental de Análise Estática de Código

Ferramenta	Propósito Arquitetural
SonarQube	Controle integrado da qualidade global, cobertura e governança do código.
PMD	Varredura e identificação automatizada de <i>code smells</i> e más práticas.
Checkstyle	Padronização estilística do código e garantia de formatação uniforme.
SpotBugs	Análise estática focada na detecção preemptiva de bugs e falhas lógicas.

20. Pipeline de Testes e Integração Contínua (CI/CD)

O fluxo de automação garante que nenhuma alteração seja implantada em produção sem passar pelos critérios rigorosos de qualidade:

21. Alinhamento com os Pilares da Clean Architecture

A estrutura proposta organiza as responsabilidades em camadas concêntricas e isoladas:

- **Camada de Domínio (*Domain Layer*):** Representada por entidades puras como `PaymentRequest` e as definições primárias das regras de negócio estruturais.
- **Camada de Aplicação (*Application Layer*):** Orquestrada pelo `PaymentService` e a definição das interfaces de comportamento (`PaymentStrategy`).
- **Camada de Infraestrutura (*Infrastructure Layer*):** Detalhes operacionais como conexões com Gateways de pagamento, bancos de dados e sistemas de mensageria (Apache Kafka).
- **Camada de Interface (*Interface Layer*):** Controladores REST expostos, mapeamentos de DTOs e documentações autogeradas via Swagger.

22. Benefícios Arquiteturais Consolidados

Abordagem Adotada	Impacto Direto no Ecossistema
Princípios SOLID	Código manutenível, legível e desacoplado.
Padrão Strategy	Inclusão simplificada de novos meios de pagamento de forma plugável.
Práticas de TDD	Segurança técnica robusta para refatorações sem regressão de bugs.
Práticas de BDD	Garantia de convergência entre os requisitos de negócio e o código técnico.
Pirâmide de Testes	Otimização do pipeline gerando ciclos de feedback rápidos e econômicos.
Clean Code	Código-fonte autoexplicativo, reduzindo custos de integração de novos desenvolvedores.
Pipeline de CI/CD	Validação automatizada contínua, blindando o ambiente de produção.

23. Linha de Evolução Histórica do EventMaster

- **Fase 1:** Decomposição da arquitetura legada (Monólito -> Microserviços).
- **Fase 2:** Introdução do modelo de processamento híbrido (Stream em Tempo Real + Processamento em Lote/Batch).
- **Fase 3:** Estabelecimento de segurança corporativa robusta com OAuth2, tokens JWT e mitigação de riscos baseada nas normas OWASP.
- **Fase 4:** Maturidade técnica por meio de SOLID, Padrões de Projeto, engenharia de testes (TDD/BDD) e governança de código.

24. Conclusão

A Fase 4 sedimenta de forma definitiva as melhores práticas e padrões de qualidade em engenharia de software para a plataforma distribuída EventMaster. O sistema eleva seu patamar de maturidade técnica demonstrando o uso eficaz de:

- Arquitetura amparada rigorosamente nos princípios SOLID
- Código limpo (*Clean Code*) com alta legibilidade
- Padrões de projeto bem aplicados (Padrão Strategy)
- Desenvolvimento seguro guiado por testes unitários com TDD
- Cobertura de comportamento endossada por ferramentas de BDD
- Definição clara de barreiras de qualidade automatizadas (*Quality Gates* via CI/CD)
- Microserviços altamente manuteníveis e resilientes
- Lógica corporativa extensível preparada para o crescimento escalável do negócio

Com a transição o protótipo inicial foi convertido em um componente maduro, modular, testável, imune a acoplamentos nocivos e tecnicamente pronto para os desafios de um ambiente produtivo corporativo de alta disponibilidade.